

3CPEs: Concrete Computational Concepts Programming Environments for Elementary Computer Science Education

Ching-Ying Sung
cysung97@gmail.com
University of Washington
Seattle, United States

Fu-Yin Cherng
fuyincherng@gmail.com
National Chung Cheng University
Chiayi County, Taiwan

Yi-Lun Chiu
i90681472a@cmlab.csie.ntu.edu.tw
National Taiwan University
Taipei, Taiwan

Peng-Hsi Chen
nonochen915@gmail.com
Keio University
Tokyo, Japan

Bing-Yu Chen
robin@ntu.edu.tw
National Taiwan University
Taipei, Taiwan

ABSTRACT

This study introduces the Concrete Computational Concepts Programming Environments (3CPEs), an innovative approach designed to help elementary students grasp abstract computational concepts through concrete metaphors using tangible and everyday objects. Traditional programming environments for children focus on concrete programming language syntax and semantics yet often fall short in conveying advanced computational concepts. To address this gap, 3CPEs combine the hands-on appeal of computer-science unplugged activities with the versatility of programming environments, using familiar objects and gamified elements to visualize abstract computational and algorithmic ideas. An example is CT (Computational Thinking) Chef, a 3CPEs implementation to enhance elementary students' understanding of programming concepts. Through design workshops with five elementary programming teachers, we preliminarily identified teaching challenges and evaluated 3CPEs' efficacy as an educational tool. The feedback underscores CT Chef's potential in facilitating the teaching of intricate computational concepts for young learners.

CCS CONCEPTS

• **Human-centered computing** → **HCI design and evaluation methods**.

KEYWORDS

Programming Environment; Concrete Metaphor Design; Computational Thinking; Elementary Education

ACM Reference Format:

Ching-Ying Sung, Fu-Yin Cherng, Yi-Lun Chiu, Peng-Hsi Chen, and Bing-Yu Chen. 2024. 3CPEs: Concrete Computational Concepts Programming Environments for Elementary Computer Science Education. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3613905.3650833>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CHI EA '24, May 11–16, 2024, Honolulu, HI, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0331-7/24/05
<https://doi.org/10.1145/3613905.3650833>

1 INTRODUCTION

In the digital age, computational thinking (CT) and computer science (CS) education are crucial in elementary education [26]. Programming environments (PEs), especially those designed for kids (PEsK), are pivotal, aiming to make abstract concepts more concrete, lowering barriers to learning [14, 25]. PEsK utilize visual block-based programming environments (VBBPEs) or text-light PEs, using metaphors for programming syntax and semantics in tools like Lego, Alice, Scratch, ScratchJR, and Kodable [6, 9, 16, 20, 23]. These have become predominant in K-12 CS education.

Despite the advantages of programming environments designed for kids, without proper guidance, students often struggle with computational and programming concepts [21]. The use of concrete representations can still lead to misconceptions about advanced concepts such as variables, loops, and conditional logic [11, 12]. Aivaloglou and Hermans' analysis of Scratch projects also indicated infrequent use of advanced concepts, revealing that existing tools might not fully support student understanding of computational concepts and algorithmic structures [1]. However, prior research has shown the importance of establishing a basic knowledge of these advanced computational concepts for the success of learning them in middle school [18]. Additionally, the K-12 Computer Science Standards [27] list several advanced computational concepts as core concepts to be introduced in grades K-2. Hence, additional support is essential to help students overcome tool limitations and develop a deeper understanding of CS concepts.

To address these challenges, teachers have incorporated alternative teaching approaches that use concrete metaphors to teach computational concepts [30, 31]. Computational concepts form the foundation of problem-solving and logical thinking in programming, encompassing skills such as creating step-by-step programs and algorithms, and utilizing data structures to store and manipulate information. Consequently, many teachers have adopted the CS Unplugged teaching strategy [29, 31], which uses interactive, digital-free activities to reinforce student understanding of computational concepts before introducing them to programming environments [30]. However, CS Unplugged activities, typically introducing only one or two concepts, lack an environment for developing multi-concept programs. This limitation makes it challenging for educators to devise exercises involving multiple concepts, which are crucial for students' holistic understanding of computer science [21]. Previous research highlights the need for a design framework

to develop suitable learning tools and interfaces for diverse education levels [21, 26, 29].

Therefore, in this study, we proposed and investigated the usability of an innovative approach called Concrete Computational Concepts Programming Environments (3CPEs). 3CPEs integrate the advantages of CS unplugged activities (e.g., concreteness and interactivity) and the flexibility of programming environments. In 3CPEs, abstract computational or algorithmic concepts are visualized using daily objects and gamified elements. Our design approach for 3CPEs involved identifying the core computational concepts in K-12 CS education, as defined by the Computer Science Teacher Association (CSTA) [27]. To design visual representations for each computational concept, we used everyday objects based on their definitions and features. To demonstrate and evaluate 3CPEs, we presented CT (Computational Thinking) Chef, an example implementation of 3CPEs designed to teach children flow-based programming concepts and skills. The current design of CT Chef, the example of 3CPEs, is versatile and can be implemented in different forms, such as a self-learning board game and tangible programming environment, offering flexibility for teachers to create various tasks and apply them in different educational settings.

Given their pivotal role in the learning process, we sought initial feedback on 3CPEs and CT Chef from teachers specializing in elementary computational thinking and programming. Teachers are essential in scaffolding for young learners and beginners to help them understand abstract learning concepts. They are also key in introducing kids' programming environments into computer science education. Their daily interactions with students allow them to refine teaching strategies, thus offering invaluable insights. Past studies emphasize the need to understand teachers' reactions to new programming environments, especially as they integrate diverse tools into K-12 computational thinking education [5, 14]. Hence, teacher involvement in assessing the feasibility and potential of 3CPEs in educational practices is critical before student evaluations. In this study, we aim to answer three key research questions (RQ): *RQ1. What difficulties and needs do elementary teachers face in teaching computational concepts? RQ2. What are teachers' attitudes about applying concreteness to computational concepts in their teaching? RQ3. How can the proposed approach, 3CPEs, be integrated into educational practices from teachers' perspectives?* To answer these questions, we hosted workshops with five elementary programming teachers, introducing 3CPEs and CT Chef, a tool that employs daily objects to visualize computational concepts. Teachers were then asked to devise teaching strategies using CT Chef's paper prototype. At the end of the workshops, we conducted semi-structured interviews and thematic analysis [22] to garner insights on their CS teaching experiences, feedback on 3CPEs, and CT Chef's usability. Findings suggest teachers found CT Chef versatile and effective in helping students comprehend complex concepts via everyday object metaphors. Based on these insights, we refined 3CPEs and CT Chef's design considerations and outlined potential improvements for future K-12 CS education support.

2 METHODOLOGY

We created CT Chef as an exemplar of 3CPEs, designed as a card-based programming tool to demonstrate the application of our

proposed approach. We chose cooking as the central theme for our initial design, but this can be adapted to suit various contexts. Our primary aim during the early design phase was to identify suitable concrete metaphors and interactive methods that would effectively integrate 3CPEs into educational practices. Consequently, we did not establish a strict progression or define content for various levels within CT Chef. The following sections detail these metaphors, associated programming rules, and their alignment with computational concepts.

2.1 Concrete Metaphor Design

The choice of concepts for CT Chef was informed by Brennan et al.'s framework [3]. The algorithm and programming concepts are derived from the K-12 CS standards of the Computer Science Teachers Association [27]. We carefully analyzed each concept's definitions and underlying meanings to mitigate superficial comprehension and misunderstanding of computational concepts. Next, with the cooking theme that CT Chef adopts, we diligently align the attributes of each computational concept with their corresponding concrete metaphors. Table 1 details each concept's definition, the corresponding metaphor designs, and illustrative images demonstrating each metaphor's representation within the card-based programming tool.

2.1.1 Conveyor Belt versus Program Sequence. Program sequence refers to the procedural relationship between instructions [27]. In CT Chef, we use the image of conveyor belts to illustrate the program sequence. As shown in the "Loop" and "Conditional" row of Table 1, users can place cards (representing instructions) on the conveyor belts to write programs and execute complex instructions, such as loops and conditional branches (e.g., if-else statements).

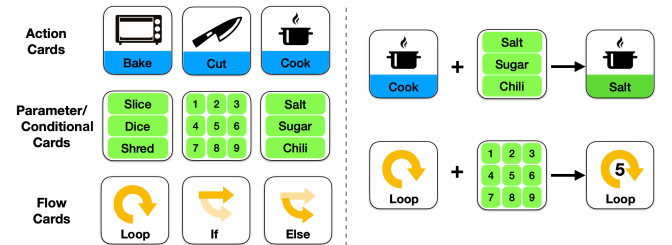


Figure 1: CT Chef operates as a card-based programming environment. Users can employ various cards, each representing different instructions, to complete a program.

2.1.2 Cards versus Instructions. As a card-based programming environment, CT Chef offers various cards as different instructions to create programs. As shown in Figure 1, the card categories include action cards, parameter cards, conditional cards, and flow cards. Action cards represent cooking actions that chefs can take, such as cutting, cooking, and stirring, which we used to represent library functions. Flow cards come in two types: loop and if-else cards. When using loop cards, the track constructed by the flow cards transforms into a circular track. This allows ingredients to follow instructions repeatedly in a cycle (e.g., "Loop" row of Table 1). Similarly, if-else cards create a branched formation, where the food

Table 1: This figure presents the definitions of various computational concepts and our metaphorical design tailored for each of these computational concepts.

Concepts	Definition of Concept	Concrete Metaphor	Image Demonstration
Program Sequence	An ordered set of instructions	Conveyor Belts	
Variable	A designated memory space within a computer program where values are stored	Food Ingredients	
Parameter	A value that is passed into a function	Parameter Card	
Loop	A sequence of statements, specified just once, that may be executed multiple times in succession	Loops Flow Card, Cyclic Tracks	
Conditional	A juncture within a computer program where the program decides its next course of action based on the outcome of a specific event	If-else Flow Card, Branch Tracks	
List	A data structure used for storing values in an ordered sequence	Ingredient Box	
Self-defined Function	A discrete segment of a program designed to perform a specific task, which can be reused throughout the program	Door, Sous Chef	

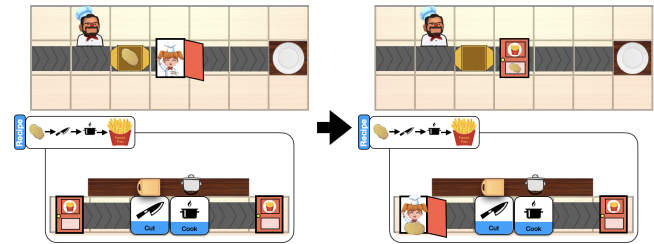
will follow one of the tracks based on the conditions specified on the if-else card. Next, parameter cards can be used with the action and flow cards to provide additional information to an instruction. For example, a parameter card can be added to a cooking card to determine which seasonings the chef would add when cooking. A numeric parameter card can be added to a loop flow card to determine how often the instructions should be repeated. Finally, conditional cards represent different food-related conditions. For instance, there may be a sliced beef card and a diced beef card.

2.1.3 Food Ingredient versus Variable. To make the concept of variables more concrete, we adopted food ingredients as metaphors. The selected ingredients are designed to be transported on a conveyor belt, undergoing alterations or processing via action cards (such as cutting or stirring) also situated on the conveyor belt. The concept of food transformation through action cards reflects how program instructions can alter variables within a program.

2.1.4 Ingredient Box versus List. We chose ingredient boxes as a metaphor for lists to illustrate the storage concept, as shown in the “List” row of Table 1. This design choice implies that a list is a collection of variables. When using lists, users must place food ingredients in the box in a specific order. If a user intends to execute a function on all the ingredients, a loop card should be used, and the executions would follow the order of the ingredient box.

2.1.5 Anywhere Door versus Self-defined Function. In CT Chef, the complex concept of self-defined functions is simplified using the “Anywhere Door” metaphor from the *Doraemon* anime series, familiar to East Asian children [33]. As shown in Figure 2, this metaphor illustrates program sequence transference, allowing users to move ingredients to different tracks with Anywhere Door cards. Additionally, reusability in self-defined functions is represented by a

sous chef inside each Anywhere Door, symbolizing the relationship between the main program (chef) and the self-defined functions (sous chef). Placing an Anywhere Door card in the sequence creates a sub-sequence with a sous chef, where users can add various cards and specify ingredients, demonstrating the function’s reusability and task specificity.

**Figure 2: The Anywhere Door concretizes self-defined functions.**

2.2 Design Workshop with CS Teachers

Drawing on qualitative research methods from prior studies [17, 29], we conducted design workshops with CS educators of elementary students. These workshops aimed to identify the challenges faced by CS educators, solicit feedback on 3CPEs, and preliminarily evaluate CT Chef’s usability. The intent was to gain detailed insights into the specific challenges and needs of CS teachers and to use this information to guide the future development of 3CPEs tools for CS education. The workshop’s design and procedures were approved by the university’s ethics committee.

Table 2: Participant’s identification number (Ps), teaching experience, materials, and information about course designs in the workshops.

Ps	Teaching Experience	Used Materials	Course Design Session	Concepts	Applied Conditions	Task Num
P1	1 year, grade 1-6	JavaScript[35], LEGO[16]	Basic	Sequence, If-else, Self-defined Function	No Specific	2
			Advanced	If-else, Conditional Loop, List	No Specific	2
P2	2.5 years, grade 1-6	Scratch[20], ScratchJR[9], MakeCode[7]	Basic	Sequence, Self-defined Function	Class Exercise	3
			Advanced	If-else	Class Exercise	2
P3	3 years, grade 7-9	Formal Language, Scratch[20]	Basic	Sequence, Loop	Class Exercise, Homework	2
			Advanced	List	Class Exercise	2
P4	1 year, grade 1-6	Coding Ocean[24], Dash[34]	Basic	Sequence, Loop	Class Exercise, Homework	2
			Advanced	List, Sequence, Self-defined Function	No Specific	2
P5	2 years, grade 1-6	Coding Ocean[24]	Basic	Sequence, Parameter	Teaching Video	4
			Advanced	Self-defined Function, If-else	Game Missions	2

2.2.1 Participants. We used snowball sampling to recruit experienced teachers in computer science, programming, and computational thinking for elementary students [10]. After selecting our participants, we conducted a series of design workshops, continuing until reaching thematic saturation [13, 15]. In total, three workshops were held with five participants (three males, two females; average age = 25), all with over a year of experience in teaching with child-friendly programming tools, including block-based and text-light programming environments and board games. Participant details are outlined in Table 2.

2.2.2 Paper Prototype of CT Chef. Paper prototyping is a widely used technique to build low-fidelity prototypes in the iterative design process. This approach allows designers to identify usability issues and improve their design in a swift and cost-effective manner [28, 32]. We constructed a paper prototype of CT Chef to facilitate participants’ assessment of its usability and feasibility, as well as to gauge their understanding of the underlying concept, 3CPEs. Participants also used the paper prototype to complete tasks we assigned during the design workshops. The paper prototype of CT Chef included paper-made conveyor belts, boxes, doors, function cards, and sticky notes that were substituted for parameter or conditional cards. Additionally, we provided blank functions and food cards, allowing participants to add new instructions and ingredients.

2.2.3 Procedure. Prior to the workshops, participants completed a pre-survey for background information, including demographics and teaching experiences. Each workshop introduced the concept of 3CPEs, the design and rules of CT Chef, and its paper prototype. Participants were tasked with creating a course using CT Chef and designing two to three related tasks. To aid in course planning, we provided course sheets for them to outline concepts, learning objectives, educational settings, and required materials, along with example recipes and brainstorming supplies. Each workshop involved two rounds of course design, focusing first on basic computational concepts like sequences and loops, and then on advanced topics such as if-else statements and self-defined functions. After finishing their course designs, participants presented their courses, shared ideas, and discussed teaching strategies. The workshops concluded with a group interview to collect feedback on CT Chef and discuss experiences in teaching computational concepts.

2.2.4 Data Collection and Analysis. Qualitative data was collected from three primary sources: participant-designed courses and tasks using CT Chef, video recordings of the workshops, and group semi-structured interviews. Course and task designs were documented via photographs and notes, while video recordings provided insight into the participants’ viewpoints. The 30-minute interviews focused on three areas: participants’ teaching experiences and approaches, their experience with CT Chef, and the influence of 3CPEs and the workshops on their teaching methods and material choices. After the workshops, we compiled 9 hours of video recordings, 10 course designs, and 23 task designs. These recordings were transcribed and analyzed using thematic analysis [2, 17, 22]. Two coders, experts in computer science and education, collaboratively identified and refined recurring themes through several discussions and cycles for accurate and reliable findings. Additionally, we cross-referenced the course design materials to gain deeper insights into the participants’ perspectives and feedback.

3 RESULTS OF COURSE AND TASK DESIGN

We showcase two designs of course provided by the participants to illustrate the diverse teaching styles in which CT Chef can be implemented to teach computational concepts. Table 2 provides an overview of the concepts and learning objectives for each course design, as well as the corresponding scenarios in which the tasks can be utilized. These examples illustrate CT Chef’s adaptability across different educational settings, such as in-class exercises, homework assignments, and teaching videos. This diversity underscores CT Chef’s capability to fit into various pedagogical approaches.

In the first design case by P1, a course teaches sequence and self-defined functions through two tasks in CT Chef, as depicted in Figure 3a. Themed around a stir-fry restaurant, students learn the roles of a chef (handling cooking actions like stirring) and a sous chef (preparing ingredients). Task 1 involves sequencing an Anywhere Door for the sous chef with action cards for the chef. Task 2 focuses on programming the sous chef’s actions within the Anywhere Door. Students use if-else cards to process ingredients differently, like cutting meat or refrigerating other items. This design effectively demonstrates practical uses of conditional logic and function definition.

In the second design case illustrated in Figure 3b, P3 focuses on teaching loop concepts in CT Chef through an in-class exercise



(a) P1's design for sequence, if-else, and self-defined function.



(b) P3's design for the concept of loop.

Figure 3: Design cases using CT Chef by P1 and P3.

(Task 1) and homework (Task 2). The course has three main objectives: identifying repeat patterns, understanding loops, and using loops to streamline programs. Task 1 involves students observing a complete program to spot repetitive sequences, aided by tracks hinting at these parts. For instance, multiple stir cards with a track above the first one can be simplified using a loop and a parameter card. Task 2 features a longer sequence with both repetitive and unique cards. This challenges students to discern repetitive patterns and apply loops appropriately, demonstrating loops' utility in simplifying programs. These course designs showcase 3CPEs' ability to teach complex concepts and adapt to different educational formats like homework and in-class exercises.

4 RESULTS OF QUALITATIVE INTERVIEW

4.1 Teaching Difficulties and Solutions

Through analysis of interview scripts, we summarized the teachers' attitudes and difficulties regarding teaching computational concepts. Firstly, P1, P2, and P3 reported that learning computational concepts and logical thinking are more important than syntax and semantics

in programming for elementary students. For example, P1 stated: "For elementary students, it is sufficient and abundant to learn the logic thinking skills and the ability to transfer knowledge of computational concepts."

Nevertheless, participants also reported difficulties in teaching computational concepts with the available materials. To address these challenges, they proposed using concrete activities to teach these concepts. For example, P4 mentioned about the struggle to teach variables and the extended concept "list": "In my experience, variables are abstract for even 7th to 9th graders. Because they could not understand why we should store a value in one thing and how we could use it."

In addition, participants reported that students often have misconceptions about setting up and recognizing a self-defined function. The hardest problem for elementary students is to predict the flows of a program when using self-defined functions. As P2 explained: "Because self-defined functions are outside the main function, they (students) could not understand why the executing thread would jump from the main function to a self-defined one and return to the main function again."

Some participants noted that their students struggled with defining conditions in conditional statements and grasping complex programming flows like nested loops. To make computational concepts more relatable, they created interactive games, including CS Unplugged activities. However, teaching multiple concepts at once proved challenging. As P3 remarked: "It is not a problem to teach children a single concept, sequences, or loops. But, if it is sequenced in a loop, they often get confused."

4.2 Feedbacks on CT Chef and 3CPEs

Based on the findings from the presentation for the designed courses and interview sessions, the participants' feedback on CT Chef and 3CPEs was positive. They noted that CT Chef was well-suited for children under eight years old, as its theme and design were relatable to their daily lives. Incorporating concreteness in the design and theme of the programming environment was also highlighted as a positive aspect. As P5 mentioned: "The cooking theme is more daily and lovely than the previous activities I prepared. The value (of CT Chef) is that it merges everyday stuff and has higher variability."

P1, P2, and P3 were satisfied with using concrete metaphors in the design, especially the conveyor belt analogy and program flow. They considered that the moving conveyor belt helped children anticipate the order and flow of a program. P1 noted: "It is a line of thought! I mean all of these belts. Students can do things and think about the orders if the line of thought can be concretized."

P2, P3, and P4 were particularly enthusiastic about the modularity metaphor in CT Chef, notably the sous-chef and Anywhere Door concepts, for teaching problem decomposition. P4 praised the design's ability to handle multiple inputs/parameters in self-defined functions, a feature not commonly found in typical programming education kits. CT Chef's versatility in accommodating various food types as inputs, as opposed to just controlling a character, was seen as beneficial for children's understanding of diverse tasks. P4 mentioned: "It becomes easy to understand when using the metaphor of sous-chef to represent modularity! Because of transitioning to the door,

they would easily know that it will get into the door and then go back to the original place. They won't feel confused about it."

Furthermore, P1, P2, and P4 expressed interest in implementing CT Chef in their future curricula. For example, P4 wanted to introduce CT Chef prior to the board game she currently uses, reasoning that the "cooking" theme is more intuitive and relevant to children than P4's existing tool. Additionally, P2 elaborated on when they would use CT Chef in their classes: "ScratchJr, for instance, is too simplistic, as it does not offer user-defined functions. Consequently, I plan to begin with ScratchJr, followed by CT Chef, and ultimately transition to Scratch."

While the cooking theme of CT Chef was generally viewed as appealing and relatable, opinions varied. P4 and P5 praised the myriad of program combinations possible within the food preparation process, stating that this could enhance students' understanding of sequencing and engagement. Conversely, P1 and P2 worried that this variability could hinder comprehension of individual concepts and result in disagreements over teacher-designed recipes, given the creative nature of cooking. They suggested limiting food ingredients and setting clear cooking process rules. P3 mentioned that limited cooking experience could hinder some students' engagement.

5 CONCLUSION AND FUTURE WORKS

Through the design workshops, our study underscores the importance of using concrete approaches in teaching computational concepts to young learners, aligning with previous studies [14, 21, 26]. Abstract concepts can confound children, and the absence of developmentally appropriate programming environments often necessitates supplementary materials to aid comprehension. Therefore, CT Chef, an exemplar of 3CPEs, emerged as a beneficial tool for introducing computational concepts to children. The findings obtained from the design workshops indicate that 3CPEs have the potential to offer elementary students a suitable environment to learn computational concepts. This is achieved by providing a low floor, which makes it easy for students to get started, and a high ceiling that creates opportunities for them to engage in increasingly complex projects. These two features of 3CPEs are beneficial for students at concrete operational developmental stages, as these features provide them with the necessary support to train their computational thinking skills without relying on abstract concepts [25].

Additionally, our study reveals the potential of 3CPEs in elementary CS education and points to the need for future designs to extend beyond concrete metaphors. Considerations should include pedagogical strategies and student developmental stages. Different learning levels may require varying approaches, such as board games and rule restrictions for beginners [4] or digital platforms and creative methods for advanced learners [19]. A collaborative effort involving designers, researchers, and educators is essential to identify educational challenges and develop appropriate techniques and environments for subsequent 3CPE designs and research.

Our research on 3CPEs has revealed potential areas for improvement and future development. In future works, we plan to examine how students' learning stages influence 3CPEs' effectiveness and explore strategies like level-designed games and free programming

support to accommodate diverse pedagogical approaches. Future research will involve collaboration with teachers to optimize learning across different contexts. Exploring the role of themes in 3CPEs is another focus, aiming to understand their impact on concretizing computational concepts and potential influences of gender or cultural disparities. Additionally, fostering peer and collaborative learning through our card-based programming environment design is a promising future direction as previous research on collaborative learning [8, 36] has demonstrated that learning computational concepts collaboratively has the potential to improve the motivation and learning outcomes of elementary students. Next, though our qualitative results offer an understanding of teachers' challenges and their feedback on 3CPEs, we recognize the need for more robust evidence. Future research should incorporate workshops and user studies involving both teachers and elementary students. By collecting quantitative data, we aim to more conclusively demonstrate the benefits of 3CPEs for both these key stakeholders. Finally, feedback from the teachers stressed the need for more advanced computational concepts in future tool versions. Hence, future designs will consider integrating concepts like additional logical operators and multiple variables to better support learners and teachers in programming education.

ACKNOWLEDGMENTS

This research was supported in part by the National Science and Technology Council of Taiwan (NSTC111-2221-E-002-145-MY3, 112-2218-E-002-029, 112-2634-F-002-006) and National Taiwan University (NTU113L894001, 113L8514, 112L900902). We also appreciate the help of participants and others involved in our study.

REFERENCES

- [1] Efthimia Aivaloglou and Felienne Hermans. 2016. How kids code and how we know: An exploratory study on the Scratch repository. In *Proceedings of the 2016 ACM Conference on International Computing Education Research*. 53–61.
- [2] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [3] Karen Brennan and Mitchel Resnick. 2012. New frameworks for studying and assessing the development of computational thinking. In *Proceedings of the 2012 annual meeting of the American educational research association, Vancouver, Canada*, Vol. 1. 25.
- [4] Ken-Zen Chen and Hsiao-Han Chi. 2022. Novice young board-game players' experience about computational thinking. *Interactive Learning Environments* 30, 8 (2022), 1375–1387.
- [5] Maria Cutumisu and Qi Guo. 2019. Using topic modeling to extract pre-service teachers' understandings of computational thinking from their coding reflections. *IEEE Transactions on Education* 62, 4 (2019), 325–332.
- [6] Wanda P Dann, Stephen Cooper, and Randy Pausch. 2011. *Learning to Program with Alice (w/CD ROM)*. Prentice Hall Press.
- [7] James Devine, Joe Finney, Peli de Halleux, Michal Moskal, Thomas Ball, and Steve Hodges. 2018. MakeCode and CODAL: intuitive and efficient embedded systems programming for education. *ACM SIGPLAN Notices* 53, 6 (2018), 19–30.
- [8] Leovy Echeverria, Ruth Cobos, and Mario Morales. 2019. Improving the students computational thinking skills with collaborative learning techniques. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje* 14, 4 (2019), 196–206.
- [9] Louise P Flannery, Brian Silverman, Elizabeth R Kazakoff, Marina Umaschi Bers, Paula Bontá, and Mitchel Resnick. 2013. Designing ScratchJr: Support for early childhood learning through computer programming. In *proceedings of the 12th international conference on interaction design and children*. 1–10.
- [10] Leo A Goodman. 1961. Snowball sampling. *The annals of mathematical statistics* (1961), 148–170.
- [11] Shuchi Grover and Satadbi Basu. 2017. Measuring student learning in introductory block-based programming: Examining misconceptions of loops, variables, and boolean logic. In *Proceedings of the 2017 ACM SIGCSE technical symposium on computer science education*. 267–272.
- [12] Shuchi Grover, Roy Pea, and Stephen Cooper. 2015. Designing for deeper learning in a blended computer science course for middle school students. *Computer science*

- education 25, 2 (2015), 199–237.
- [13] Monique M Hennink, Bonnie N Kaiser, and Vincent C Marconi. 2017. Code saturation versus meaning saturation: how many interviews are enough? *Qualitative health research* 27, 4 (2017), 591–608.
 - [14] Charlotte Hill, Hilary A Dwyer, Tim Martinez, Danielle Harlow, and Diana Franklin. 2015. Floors and Flexibility: Designing a programming environment for 4th–6th grade classrooms. In *Proceedings of the 46th ACM technical Symposium on computer science education*. 546–551.
 - [15] Cicely Kerr, Annabel Nixon, and Diane Wild. 2010. Assessing and demonstrating data saturation in qualitative inquiry supporting patient-reported outcomes research. *Expert review of pharmacoeconomics & outcomes research* 10, 3 (2010), 269–281.
 - [16] Serhat Bahadır Kert, Mehmet Fatih Erkoç, and Sabiha Yeni. 2020. The Effect of Robotics on Six Graders' Academic Achievement, Computational Thinking Skills and Conceptual Knowledge Levels. *Thinking Skills and Creativity* (2020), 100714.
 - [17] Jac Ka Lok Leung, Samuel Kai Wah Chu, Ting-Chuen Pong, Davy Tsz Kit Ng, and Shen Qiao. 2021. Developing a framework for blended design-based learning in a first-year multidisciplinary design course. *IEEE Transactions on Education* 65, 2 (2021), 210–219.
 - [18] Feiya Luo, Wei Yan, Ruohan Liu, and Maya Israel. 2022. Elementary Students' Understanding of Variables in Computational Thinking-Integrated Instruction: A Mixed Methods Study. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education-Volume 1*. 523–529.
 - [19] Bronwyn MacFarlane. 2021. Infrastructure of comprehensive STEM programming for advanced learners. In *STEM Education for High-Ability Learners*. Routledge, 139–160.
 - [20] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The scratch programming language and environment. *ACM Transactions on Computing Education (TOCE)* 10, 4 (2010), 1–15.
 - [21] Rodrigo Pessoa Medeiros, Geber Lisboa Ramalho, and Taciana Pontual Falcão. 2018. A systematic literature review on teaching and learning introductory programming in higher education. *IEEE Transactions on Education* 62, 2 (2018), 77–90.
 - [22] Matthew B Miles and A Michael Huberman. 1994. *Qualitative data analysis: An expanded sourcebook*. sage.
 - [23] L Orsini. 2013. Kodable Teaches Kids To Code Before They Learn To Read. *Recuperado a partir de* <http://readwrite.com/2013/04/23/kodable-teaches-kids-to-codebefore-they-learn-to-read#awesm=^oDmcKub1ryv8TN> (2013).
 - [24] Papacode. 2020. I want to buy programming education board game, Coding Ocean! – Papacode. <http://www.papacode.com.tw/codingocean.html> [Online; accessed 17-September-2020].
 - [25] Mitchel Resnick and Brian Silverman. 2005. Some reflections on designing construction kits for kids. In *Proceedings of the 2005 conference on Interaction design and children*. 117–122.
 - [26] Sara Rijo-García, Eduardo Segredo, and Coromoto León. 2022. Computational thinking and user interfaces: A systematic review. *IEEE Transactions on Education* (2022).
 - [27] Deborah Seehorn, Tammy Pirmann, Todd Lash, Bryan Twarek, Daniel Moix, Leticia Batista, Julia Bell, Chris Kuszmaul, Dianne O'Grady-Cuniff, Minsoo Park, Lori Pollock, Meg Ray, Dylan Ryder, Vicky Sedgwick, Grant Smith, and Chinma Uche. 2017. *CSTA K–12 Computer Science Standards: Revised 2017*. ACM.
 - [28] Reinhard Sefelin, Manfred Tscheligi, and Verena Giller. 2003. Paper prototyping-what is it good for? A comparison of paper-and computer-based low-fidelity prototyping. In *CHI'03 extended abstracts on Human factors in computing systems*. 778–779.
 - [29] Ivana Storjak, Ana Sovic Krzic, and Tomislav Jagust. 2022. Elementary School Pupils' Mental Models Regarding Robots and Programming. *IEEE Transactions on Education* 65, 3 (2022), 297–308.
 - [30] Anthony Trory, Kate Howland, and Judith Good. 2018. Designing for concreteness fading in primary computing. In *Proceedings of the 17th ACM Conference on Interaction Design and Children*. 278–288.
 - [31] CS Unplugged. [n.d.]. *CS Unplugged*. <https://csunplugged.org/en/>.
 - [32] Miriam Walker, Leila Takayama, and James A Landay. 2002. High-fidelity or low-fidelity, paper or computer? Choosing attributes when testing web prototypes. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 46. Sage Publications Sage CA: Los Angeles, CA, 661–665.
 - [33] Wikipedia. 2023. Doraemon Anywhere Door—Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/wiki/Doraemon#Gadgets> [Online; accessed 30-May-2023].
 - [34] Inc Wonder Workshop. 2020. Dash - Wonder Workshop - US. <https://www.makewonder.com/robots/dash/> [Online; accessed 17-September-2020].
 - [35] Penn Wu. 2009. Teaching basic game programming using javascript. *Journal of Computing Sciences in Colleges* 24, 4 (2009), 211–220.
 - [36] María Zapata-Cáceres, Estefanía Martín-Barroso, and Marcos Román-González. 2021. Collaborative game-based environment and assessment tool for learning computational thinking in primary school: a case study. *IEEE Transactions on Learning Technologies* 14, 5 (2021), 576–589.